

Using React Native, build across platform application for a BMI calculator.

Aim:-

To create an app using react native, build across platform application for a BMI calculator.

Procedure

Step 1: Verify Node.js Installation

First, check if Node.js is already installed on your system:

```
node --version
```

```
npm --version
```

If these commands show version numbers (Node.js v18 or later), you can skip to Step 2.

Step 2: Install Node.js (if not installed):

```
winget install OpenJS.NodeJS
```

Install Expo CLI globally on your system:

```
npm install -g @expo/cli
```

If you get PowerShell execution policy error:

```
# Run this first, then retry npm install
```

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Step 3: Verify Expo installation:

```
expo --version
```

Step 4: Create New Expo Project

```
# Navigate to exercise folder
```

```
cd exercise-1-bmi-calculator
```

```
# Install dependencies
```

```
npm install
```

Step 5: Install Required Dependencies

```
npm install react-native-paper
```

Step 6: Replace Default Code

- Navigate to app/(tabs)/ folder
- Replace the content of index.tsx with the BMI calculator code

Running the Application

Step 7: Start Development Server

Open terminal in your project directory and run:

```
npx expo start
```

Run in Web Browser:

```
npx expo start --web
```

Program

Index.tsx

```
import React, { useState } from 'react';
import { SafeAreaView, View, StyleSheet, KeyboardAvoidingView, Platform } from
'react-native';
import { Provider as PaperProvider, TextInput, Button, Text, Title, Card } from 'react-
native-paper';
export default function App() {
  const [height, setHeight] = useState<string>("");
  const [weight, setWeight] = useState<string>("");
  const [bmi, setBmi] = useState<number | null>(null);
  const [category, setCategory] = useState<string>("");
  const calculateBMI = () => {
    const h = parseFloat(height) / 100;
    const w = parseFloat(weight);

    if (isNaN(h) || isNaN(w) || h <= 0 || w <= 0) {
      setBmi(null);
      setCategory('Please enter valid height and weight');
      return;
    }

    const bmiValue = parseFloat((w / (h * h)).toFixed(1));
    setBmi(bmiValue);

    if (bmiValue < 18.5) setCategory('Underweight');
    else if (bmiValue < 24.9) setCategory('Normal weight');
    else if (bmiValue < 29.9) setCategory('Overweight');
    else setCategory('Obese');
  };
  return (
    <PaperProvider>
```

```

<SafeAreaView style={styles.container}>
  <KeyboardAvoidingView
    style={{ flex: 1 }}
    behavior={Platform.OS === 'ios' ? 'padding' : undefined}
  >
    <View style={styles.innerContainer}>
      <Title style={styles.title}>BMI Calculator</Title>

      <TextInput
        label="Height (cm)"
        value={height}
        onChangeText={setHeight}
        keyboardType="numeric"
        style={styles.input}
        mode="outlined"
        placeholder="e.g. 170"
      />

      <TextInput
        label="Weight (kg)"
        value={weight}
        onChangeText={setWeight}
        keyboardType="numeric"
        style={styles.input}
        mode="outlined"
        placeholder="e.g. 65"
      />

      <Button mode="contained" onPress={calculateBMI} style={styles.button}>
        Calculate BMI
      </Button>

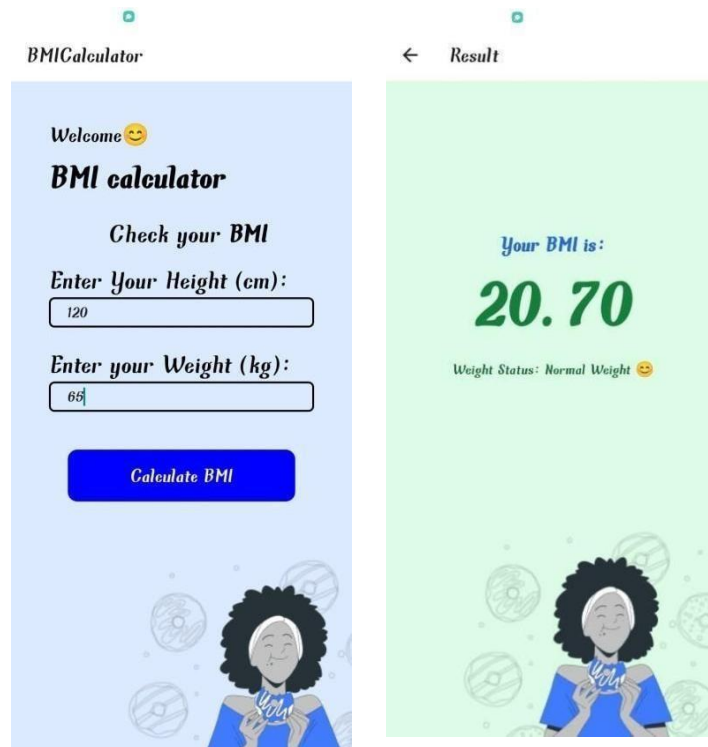
      {bmi !== null && (
        <Card style={styles.resultCard}>
          <Card.Content>
            <Title>Your BMI: {bmi}</Title>
            <Text>Status: {category}</Text>
          </Card.Content>
        </Card>
      )}

      {bmi === null && category !== " " && (
        <Text style={styles.errorText}>{category}</Text>
      )}
    </View>
  </KeyboardAvoidingView>
</SafeAreaView>
</PaperProvider>
);
}

```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#e81515ca',
  },
  innerContainer: {
    flex: 1,
    justifyContent: 'center',
    paddingHorizontal: 20,
  },
  title: {
    alignSelf: 'center',
    marginBottom: 30,
  },
  input: {
    marginBottom: 15,
  },
  button: {
    marginTop: 10,
  },
  resultCard: {
    marginTop: 30,
    backgroundColor: '#E0F7FA',
  },
  errorText: {
    marginTop: 20,
    color: 'red',
    textAlign: 'center',
  },
});
```

Output



The image displays two screenshots of a BMI calculator application. The left screenshot, titled "BMICalculator", shows the input screen with a light blue background. It includes a "Welcome" message with a smiley face, the title "BMI calculator", and the instruction "Check your BMI". There are two input fields: "Enter Your Height (cm):" with the value "120" and "Enter your Weight (kg):" with the value "65". A blue button labeled "Calculate BMI" is at the bottom. The right screenshot, titled "Result", shows the output screen with a light green background. It displays "Your BMI is: 20.70" in large green text, followed by "Weight Status: Normal Weight" with a smiley face. Both screens feature a cartoon illustration of a woman with dark curly hair and a blue top at the bottom.

Result:

Thus the BMI calculator app was developed and executed successfully.

	Build a cross platform application for a simple expense manager which allows Entering expenses and income on each day and displays categorywise weekly income and expense.
--	---

Aim:-

To Build across platform application for a simple expense manager which allows entering expenses and income on each day and displays category wise weekly income and expense.

Procedure:-**Step 1: Verify Node.js Installation**

First, check if Node.js is already installed on your system:

```
node --version
```

npm --version If these commands show version numbers (Node.js v18 or later), you can skip to Step 2.

Install Node.js (if not installed):

```
winget install OpenJS.NodeJS
```

Step 2: Install Expo CLI

Install Expo CLI globally on your system:

```
npm install -g @expo/cli
```

If you get PowerShell execution policy error

```
:# Run this first, then retry npm install
```

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Step 3: Verify Expo installation:

```
expo --version
```

Step 4: Create New Expo Project

```
# Navigate to exercise folder
```

```
cd exercise-2-expense-manager
```

```
# Install dependencies
```

```
npm install
```

Step 5: Install Required Dependencies

```
npm install react-native-paper
```

Step 6: Replace/Add Files**Running the Application****Step 7: Start Development Server**

Open terminal in your project directory and run:

```
expo start
```

Program

Index.tsx

```
import React from 'react';
import { View, StyleSheet, ScrollView, ActivityIndicator } from 'react-native';
import { Text, Button, useTheme, Card, IconButton } from 'react-native-paper';
import { SummaryCard } from '../components/SummaryCard';
import { TransactionCard } from '../components/TransactionCard';
import { useEffect, useState } from 'react';
import { Transaction } from '../utils/types';
import {
  getCurrentWeekTransactions,
  deleteTransaction
} from '../utils/storage';
import { router } from 'expo-router';

export default function Dashboard() {
  const theme = useTheme();
  const [transactions, setTransactions] = useState<Transaction[]>([]);
  const [loading, setLoading] = useState<boolean>(true);

  useEffect(() => {
    loadTransactions();
  }, []);

  const loadTransactions = async () => {
    setLoading(true);
    try {
      const weeklyTransactions = await getCurrentWeekTransactions();
      // Sort by date, newest first
      weeklyTransactions.sort((a, b) =>
        new Date(b.date).getTime() - new Date(a.date).getTime()
      );
      setTransactions(weeklyTransactions);
    } catch (error) {
      console.error("Failed to load transactions", error);
    } finally {
      setLoading(false);
    }
  };

  const handleDelete = async (id: string) => {
    try {
      await deleteTransaction(id);
      // Refresh transactions after delete
      loadTransactions();
    } catch (error) {
      console.error("Failed to delete transaction", error);
    }
  };
};
```

```

const handleAddTransaction = () => {
  router.push('/add-transaction');
};

const handleRefresh = () => {
  loadTransactions();
};

if (loading) {
  return (
    <View style={styles.loadingContainer}>
      <ActivityIndicator size="large" color={theme.colors.primary} />
      <Text style={styles.loadingText}>Loading transactions...</Text>
    </View>
  );
}

return (
  <View style={styles.container}>
    <SummaryCard transactions={transactions} />

    <View style={styles.header}>
      <Text variant="titleLarge" style={styles.title}>
        Recent Transactions
      </Text>
      <IconButton
        icon="refresh"
        size={20}
        onPress={handleRefresh}
        iconColor={theme.colors.primary}
      />
    </View>

    <ScrollView style={styles.transactionsList}>
      {transactions.length === 0 ? (
        <Card style={styles.emptyCard}>
          <Card.Content style={styles.emptyContent}>
            <Text variant="bodyLarge" style={styles.emptyText}>
              No transactions this week
            </Text>
            <Button
              mode="contained"
              onPress={handleAddTransaction}
              style={styles.addButton}
            >
              Add Transaction
            </Button>
          </Card.Content>
        </Card>
      )}
    </ScrollView>
  </View>
);

```



```

    ): (
      transactions.map(transaction => (
        <TransactionCard
          key={transaction.id}
          transaction={transaction}
          onDelete={handleDelete}
        />
      ))
    )}
  </ScrollView>

  {transactions.length > 0 && (
    <Button
      mode="contained"
      onPress={handleAddTransaction}
      style={styles.floatingButton}
      icon="plus"
    >
      Add Transaction
    </Button>
  )}
</View>
);
}

```

```

const styles = StyleSheet.create({
  container: {
    flex: 1,
    paddingHorizontal: 16,
    paddingTop: 16,
    backgroundColor: '#f5f5f5',
  },
  loadingContainer: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
  loadingText: {
    marginTop: 10,
  },
  header: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginVertical: 8,
  },
  title: {
    fontWeight: 'bold',
  },
  transactionsList: {

```

```

    flex: 1,
    marginBottom: 70,
  },
  emptyCard: {
    marginVertical: 20,
    elevation: 2,
  },
  emptyContent: {
    alignItems: 'center',
    paddingVertical: 20,
  },
  emptyText: {
    marginBottom: 16,
    opacity: 0.6,
  },
  addButton: {
    marginTop: 10,
  },
  floatingButton: {
    position: 'absolute',
    bottom: 20,
    right: 16,
    left: 16,
  },
});

```

Reports.tsx

```

import React, { useEffect, useState } from 'react';
import { View, StyleSheet, ScrollView, Dimensions } from 'react-native';
import { Text, Card, useTheme, Button } from 'react-native-paper';
import {
  getTransactions,
  getTransactionsByDateRange,
  getCategorySummary,
  calculateTotalExpenses,
  calculateTotalIncome,
  getCategories
} from '../utils/storage';
import { Transaction, Category } from '../utils/types';
import {
  startOfWeek,
  endOfWeek,
  format,
  subWeeks,
  addWeeks
} from 'date-fns';
import { LineChart, PieChart } from 'react-native-chart-kit';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';

```

```

export default function Reports() {

```

```

const theme = useTheme();
const [transactions, setTransactions] = useState<Transaction[]>([]);
const [categories, setCategories] = useState<Category[]>([]);
const [weekStart, setWeekStart] = useState(startOfWeek(new Date()));
const [weekEnd, setWeekEnd] = useState(endOfWeek(new Date()));
const [loading, setLoading] = useState(true);

const screenWidth = Dimensions.get('window').width - 32; // Account for padding

useEffect(() => {
  loadData();
}, [weekStart, weekEnd]);

const loadData = async () => {
  setLoading(true);
  try {
    const weeklyTransactions = await getTransactionsByDateRange(weekStart,
weekEnd);
    const allCategories = await getCategories();

    setTransactions(weeklyTransactions);
    setCategories(allCategories);
  } catch (error) {
    console.error("Failed to load data", error);
  } finally {
    setLoading(false);
  }
};

const previousWeek = () => {
  setWeekStart(subWeeks(weekStart, 1));
  setWeekEnd(subWeeks(weekEnd, 1));
};

const nextWeek = () => {
  setWeekStart(addWeeks(weekStart, 1));
  setWeekEnd(addWeeks(weekEnd, 1));
};

const currentWeek = () => {
  setWeekStart(startOfWeek(new Date()));
  setWeekEnd(endOfWeek(new Date()));
};

const getCategoryColor = (categoryName: string) => {
  const category = categories.find(cat => cat.name === categoryName);
  return category?.color || '#999999';
};

const expensesByCategory = getCategorySummary(transactions);

```

```

const totalExpenses = calculateTotalExpenses(transactions);
const totalIncome = calculateTotalIncome(transactions);

const pieChartData = Object.entries(expensesByCategory).map(([name, amount])
=> ({
  name,
  amount,
  color: getCategoryColor(name),
  legendFontColor: '#7F7F7F',
  legendFontSize: 12,
})));

// Extract daily totals for line chart
const dailyData = () => {
  const days = Array.from({ length: 7 }, (_, i) => {
    const day = addWeeks(weekStart, 0);
    day.setDate(weekStart.getDate() + i);
    return {
      date: day,
      dayString: format(day, 'EEE'),
      expenses: 0,
      income: 0,
    };
  });

  transactions.forEach(transaction => {
    const date = new Date(transaction.date);
    const dayIndex = Math.floor((date.getTime() - weekStart.getTime()) / (24 * 60 *
60 * 1000));

    if (dayIndex >= 0 && dayIndex < 7) {
      if (transaction.type === 'expense') {
        days[dayIndex].expenses += transaction.amount;
      } else {
        days[dayIndex].income += transaction.amount;
      }
    }
  });

  return days;
};

const chartData = {
  labels: dailyData().map(d => d.dayString),
  datasets: [
    {
      data: dailyData().map(d => d.expenses),
      color: () => 'rgba(220, 53, 69, 0.8)',
      strokeWidth: 2,
    },
  ],

```

```

    {
      data: dailyData().map(d => d.income),
      color: () => 'rgba(40, 167, 69, 0.8)',
      strokeWidth: 2,
    },
  ],
  legend: ["Expenses", "Income"]
};

const chartConfig = {
  backgroundGradientFrom: '#FFFFFF',
  backgroundGradientTo: '#FFFFFF',
  decimalPlaces: 0,
  color: (opacity = 1) => `rgba(0, 0, 0, ${opacity})`,
  labelColor: (opacity = 1) => `rgba(0, 0, 0, ${opacity})`,
  style: {
    borderRadius: 16,
  },
  propsForDots: {
    r: '5',
    strokeWidth: '1',
  },
};

return (
  <SafeAreaView style={styles.container}>
    <ScrollView>
      <Card style={styles.weekSelector}>
        <Card.Content style={styles.weekSelectorContent}>
          <Button onPress={previousWeek} mode="text">
            <Ionicons name="chevron-back" size={18} />
          </Button>
          <Text variant="titleMedium">
            {format(weekStart, 'MMM d')} - {format(weekEnd, 'MMM d, yyyy')}
          </Text>
          <Button onPress={nextWeek} mode="text">
            <Ionicons name="chevron-forward" size={18} />
          </Button>
        </Card.Content>
        <Card.Actions style={styles.weekSelectorActions}>
          <Button onPress={currentWeek} mode="outlined">
            Current Week
          </Button>
        </Card.Actions>
      </Card>

      <Card style={styles.summaryCard}>
        <Card.Content>
          <Text variant="titleMedium" style={styles.cardTitle}>Weekly
Summary</Text>

```

```

<View style={styles.summaryRow}>
  <Text variant="bodyLarge">Income:</Text>
  <Text
    variant="bodyLarge"
    style={{ color: '#28a745', fontWeight: 'bold' }}
  >
    ${totalIncome.toFixed(2)}
  </Text>
</View>
<View style={styles.summaryRow}>
  <Text variant="bodyLarge">Expenses:</Text>
  <Text
    variant="bodyLarge"
    style={{ color: '#dc3545', fontWeight: 'bold' }}
  >
    ${totalExpenses.toFixed(2)}
  </Text>
</View>
<View style={styles.summaryRow}>
  <Text variant="bodyLarge">Balance:</Text>
  <Text
    variant="bodyLarge"
    style={{
      color: totalIncome - totalExpenses >= 0 ? '#28a745' : '#dc3545',
      fontWeight: 'bold'
    }}
  >
    ${(totalIncome - totalExpenses).toFixed(2)}
  </Text>
</View>
</Card.Content>
</Card>

<Card style={styles.chartCard}>
  <Card.Content>
    <Text variant="titleMedium" style={styles.cardTitle}>Daily
Transactions</Text>
    {transactions.length > 0 ? (
      <LineChart
        data={chartData}
        width={screenWidth - 40}
        height={220}
        chartConfig={chartConfig}
        bezier
        style={styles.chart}
      />
    ) : (
      <View style={styles.noDataContainer}>
        <Text variant="bodyMedium" style={styles.noDataText}>No data for this
period</Text>

```

```

        </View>
    })
</Card.Content>
</Card>

<Card style={styles.chartCard}>
  <Card.Content>
    <Text variant="titleMedium" style={styles.cardTitle}>Expenses by
Category</Text>
    {Object.keys(expensesByCategory).length > 0 ? (
      <PieChart
        data={pieChartData}
        width={screenWidth - 40}
        height={200}
        chartConfig={chartConfig}
        accessor="amount"
        backgroundColor="transparent"
        paddingLeft="10"
        center={[10, 0]}
        absolute
      />
    ) : (
      <View style={styles.noDataContainer}>
        <Text variant="bodyMedium" style={styles.noDataText}>No expense data
for this period</Text>
      </View>
    )}
  </Card.Content>
</Card>

<Card style={[styles.chartCard, { marginBottom: 40 }]}>
  <Card.Content>
    <Text variant="titleMedium" style={styles.cardTitle}>Expense
Breakdown</Text>
    {Object.keys(expensesByCategory).length > 0 ? (
      Object.entries(expensesByCategory)
        .sort(([, a], [, b]) => b - a) // Sort by amount, descending
        .map(([, category, amount]) => (
          <View key={category} style={styles.categoryRow}>
            <View style={styles.categoryInfo}>
              <View
                style={[
                  styles.colorIndicator,
                  { backgroundColor: getCategoryColor(category) }
                ]}
              />
              <Text variant="bodyMedium">{category}</Text>
            </View>
            <View style={styles.amountContainer}>
              <Text variant="bodyMedium">${amount.toFixed(2)}</Text>
            </View>
          </View>
        )
      ) : (
        <Text variant="bodyMedium">No expense data for this period</Text>
      )
    )}
  </Card.Content>
</Card>

```

```

        <Text variant="bodySmall" style={styles.percentage}>
          {((amount / totalExpenses) * 100).toFixed(0)}%
        </Text>
      </View>
    </View>
  ))
) : (
  <View style={styles.noDataContainer}>
    <Text variant="bodyMedium" style={styles.noDataText}>No expenses for
this period</Text>
  </View>
  )}
</Card.Content>
</Card>
</ScrollView>
</SafeAreaView>
);
}

```

```

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f5f5f5',
    padding: 16,
  },
  weekSelector: {
    marginBottom: 16,
  },
  weekSelectorContent: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
  },
  weekSelectorActions: {
    justifyContent: 'center',
  },
  summaryCard: {
    marginBottom: 16,
  },
  chartCard: {
    marginBottom: 16,
    elevation: 2,
  },
  cardTitle: {
    fontWeight: 'bold',
    marginBottom: 16,
    textAlign: 'center',
  },
  summaryRow: {
    flexDirection: 'row',

```



```

      justifyContent: 'space-between',
      marginBottom: 8,
    },
    chart: {
      marginVertical: 8,
      borderRadius: 16,
    },
    noDataContainer: {
      height: 150,
      justifyContent: 'center',
      alignItems: 'center',
    },
    noDataText: {
      opacity: 0.5,
    },
    categoryRow: {
      flexDirection: 'row',
      justifyContent: 'space-between',
      alignItems: 'center',
      paddingVertical: 10,
      borderBottomWidth: 1,
      borderBottomColor: '#e0e0e0',
    },
    categoryInfo: {
      flexDirection: 'row',
      alignItems: 'center',
      flex: 1,
    },
    colorIndicator: {
      width: 12,
      height: 12,
      borderRadius: 6,
      marginRight: 8,
    },
    amountContainer: {
      alignItems: 'flex-end',
    },
    percentage: {
      opacity: 0.6,
      marginTop: 2,
    },
  },
});

```

Settings.tsx

```

import React, { useEffect, useState } from 'react';
import { View, StyleSheet, Alert, ScrollView } from 'react-native';
import { Text, List, Button, Divider, Dialog, Portal, useTheme } from 'react-native-paper';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { SafeAreaView } from 'react-native-safe-area-context';

```

```

export default function Settings() {
  const theme = useTheme();
  const [showClearDialog, setShowClearDialog] = useState(false);
  const [totalStorage, setTotalStorage] = useState('0 KB');

  useEffect(() => {
    calculateStorageUsage();
  }, []);

  const calculateStorageUsage = async () => {
    try {
      const keys = await AsyncStorage.getAllKeys();
      let totalSize = 0;

      for (const key of keys) {
        if (key.startsWith('expense_manager_')) {
          const data = await AsyncStorage.getItem(key);
          if (data) {
            totalSize += new Blob([data]).size;
          }
        }
      }

      // Convert bytes to KB
      const sizeInKB = (totalSize / 1024).toFixed(2);
      setTotalStorage(`${sizeInKB} KB`);
    } catch (error) {
      console.error("Failed to calculate storage usage", error);
    }
  };

  const clearAllData = async () => {
    try {
      const keys = await AsyncStorage.getAllKeys();
      const expenseManagerKeys = keys.filter(key =>
key.startsWith('expense_manager_'));

      if (expenseManagerKeys.length > 0) {
        await AsyncStorage.multiRemove(expenseManagerKeys);
        Alert.alert(
          "Success",
          "All expense data has been cleared",
          [{ text: "OK" }]
        );
        setTotalStorage('0 KB');
      }
    } catch (error) {
      console.error("Failed to clear data", error);
      Alert.alert(
        "Error",

```

```

        "Failed to clear data. Please try again.",
        [{ text: "OK" }]
    );
    } finally {
        setShowClearDialog(false);
    }
};

const exportData = async () => {
    try {
        const keys = await AsyncStorage.getAllKeys();
        const expenseManagerKeys = keys.filter(key =>
key.startsWith('expense_manager_'));

        const exportData = {};
        for (const key of expenseManagerKeys) {
            const data = await AsyncStorage.getItem(key);
            if (data) {
                exportData[key] = JSON.parse(data);
            }
        }

        Alert.alert(
            "Export Data",
            "This is a demo feature. In a real app, this would save your data to a file or cloud
service.",
            [{ text: "OK" }]
        );

        console.log("Export data:", JSON.stringify(exportData, null, 2));
    } catch (error) {
        console.error("Failed to export data", error);
        Alert.alert("Error", "Failed to export data");
    }
};

const importData = () => {
    Alert.alert(
        "Import Data",
        "This is a demo feature. In a real app, this would let you import data from a file or
cloud service.",
        [{ text: "OK" }]
    );
};

return (
    <SafeAreaView style={styles.safeArea}>
        <ScrollView style={styles.container}>
            <Text variant="headlineMedium" style={styles.title}>Settings</Text>

```

```

<List.Section>
  <List.Subheader>Data Management</List.Subheader>
  <List.Item
    title="Storage Usage"
    description={`App data is using ${totalStorage}`}
    left={props => <List.Icon {...props} icon="database" />}
  />
  <Divider />
  <List.Item
    title="Export Data"
    description="Save your expense data"
    left={props => <List.Icon {...props} icon="export" />}
    onPress={exportData}
  />
  <Divider />
  <List.Item
    title="Import Data"
    description="Load previously exported data"
    left={props => <List.Icon {...props} icon="import" />}
    onPress={importData}
  />
  <Divider />
  <List.Item
    title="Clear All Data"
    description="Delete all transactions and categories"
    left={props => <List.Icon {...props} icon="delete" color={theme.colors.error}
  />}
  <List.Item
    title="Clear All Data"
    description="Delete all transactions and categories"
    left={props => <List.Icon {...props} icon="delete" color={theme.colors.error}
    onPress={() => setShowClearDialog(true)}
  />
</List.Section>

<List.Section>
  <List.Subheader>About</List.Subheader>
  <List.Item
    title="Version"
    description="1.0.0"
    left={props => <List.Icon {...props} icon="information" />}
  />
  <Divider />
  <List.Item
    title="Exercise 2: Expense Manager"
    description="App Development Laboratory (CCS332)"
    left={props => <List.Icon {...props} icon="school" />}
  />
</List.Section>

<Portal>
  <Dialog visible={showClearDialog} onDismiss={() =>
setShowClearDialog(false)}>
    <Dialog.Title>Clear All Data</Dialog.Title>

```

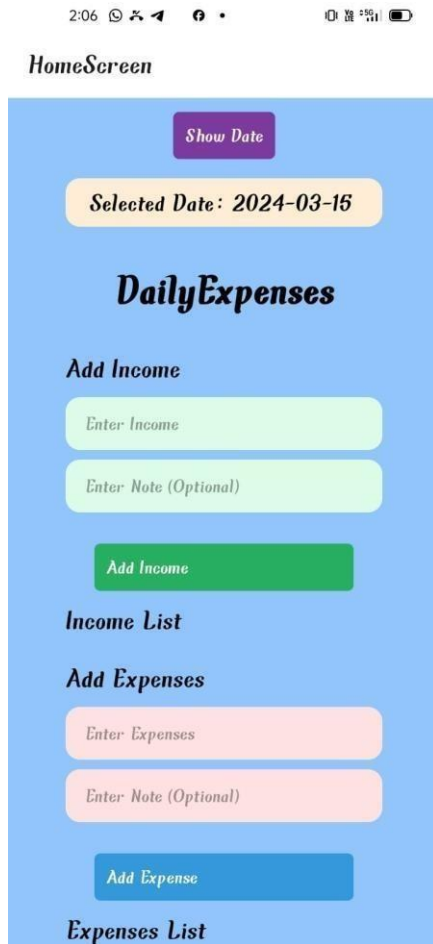
```

        <Dialog.Content>
          <Text variant="bodyMedium">
            Are you sure you want to clear all expense data? This action cannot be
undone.
          </Text>
        </Dialog.Content>
        <Dialog.Actions>
          <Button onPress={() => setShowClearDialog(false)}>Cancel</Button>
          <Button onPress={clearAllData} textColor={theme.colors.error}>Clear
All</Button>
        </Dialog.Actions>
      </Dialog>
    </Portal>
  </ScrollView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
  },
  container: {
    flex: 1,
    backgroundColor: '#f5f5f5',
    padding: 16,
  },
  title: {
    fontWeight: 'bold',
    marginBottom: 24,
  },
});

```

OUTPUT



Result:

Thus the Expense Manager app was developed and executed successfully.

Develop a cross platform application to convert units from imperial system to metric system (Km to miles, kg to pounds etc.,)
--

Aim:-

To develop a cross platform application to convert units from imperial system to metric system (km to miles, kg to pounds etc.,)

Procedure:-

Flutter Installation

Download flutter SDK from the official site
<https://flutter.dev/docs/get-started/install/windows>

Download the latest Flutter SDK ZIP.

Extract it to a folder (e.g., C:\src\flutter).

Add Flutter to PATH

Open Start → Edit the system environment variables → Environment Variables.

Under “User variables”, find or create Path → Click Edit → New.

Add the path:C:\src\flutter\bin

Click OK to save.

Verify Installation

Open Command Prompt and run:

```
flutter doctor
```

This checks your environment and displays any missing dependencies.

open vs code :

```
flutter create unit_converter
```

```
cd unit_converter
```

```
code .
```

change the code in the lib/main.dart

To run the application

```
flutter run
```

Program

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

void main() {
  runApp(const UnitConverterApp());
}

class UnitConverterApp extends StatelessWidget {
  const UnitConverterApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Unit Converter',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
      ),
      home: const HomeScreen(),
    );
  }
}

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  int _selectedIndex = 0;

  final List<Widget> _pages = const [
    ConverterPage(
      title: "Length Converter",
      conversions: ["Km → Miles", "Miles → Km"],
    ),
    ConverterPage(
      title: "Weight Converter",
      conversions: ["Kg → Pounds", "Pounds → Kg"],
    ),
    ConverterPage(
      title: "Temperature Converter",
      conversions: ["°C → °F", "°F → °C"],
    ),
    ConverterPage(
      title: "Volume Converter",
```



```

        conversions: ["Liters → Gallons", "Gallons → Liters"],
    ),
];

@override
Widget build(BuildContext context) {
    return Scaffold(
        body: _pages[_selectedIndex],
        bottomNavigationBar: BottomNavigationBar(
            currentIndex: _selectedIndex,
            selectedItemColor: Colors.indigo,
            unselectedItemColor: Colors.grey,
            type: BottomNavigationBarType.fixed,
            onTap: (index) {
                setState() {
                    _selectedIndex = index;
                });
            },
            items: const [
                BottomNavigationBarItem(icon: Icon(Icons.straighten), label: "Length"),
                BottomNavigationBarItem(icon: Icon(Icons.monitor_weight), label: "Weight"),
                BottomNavigationBarItem(icon: Icon(Icons.thermostat), label: "Temp"),
                BottomNavigationBarItem(icon: Icon(Icons.local_drink), label: "Volume"),
            ],
        ),
    );
}

```

```

class ConverterPage extends StatefulWidget {
    final String title;
    final List<String> conversions;

```

```

    const ConverterPage({
        super.key,
        required this.title,
        required this.conversions,
    });

```

```

    @override
    State<ConverterPage> createState() => _ConverterPageState();
}

```

```

class _ConverterPageState extends State<ConverterPage> {
    final TextEditingController _inputController = TextEditingController();
    late String _selectedConversion;
    String _result = "";

```

```

    @override
    void initState() {

```

```

super.initState();
_selectedConversion = widget.conversions.first;
}

void _convert() {
  final input = double.tryParse(_inputController.text);
  if (input == null) {
    setState(() {
      _result = " Enter a valid number!";
    });
    return;
  }

  double output;
  switch (_selectedConversion) {
    case "Km → Miles":
      output = input * 0.621371;
      _result = "$input km = ${output.toStringAsFixed(2)} miles";
      break;
    case "Miles → Km":
      output = input / 0.621371;
      _result = "$input miles = ${output.toStringAsFixed(2)} km";
      break;
    case "Kg → Pounds":
      output = input * 2.20462;
      _result = "$input kg = ${output.toStringAsFixed(2)} lbs";
      break;
    case "Pounds → Kg":
      output = input / 2.20462;
      _result = "$input lbs = ${output.toStringAsFixed(2)} kg";
      break;
    case "°C → °F":
      output = (input * 9 / 5) + 32;
      _result = "$input °C = ${output.toStringAsFixed(2)} °F";
      break;
    case "°F → °C":
      output = (input - 32) * 5 / 9;
      _result = "$input °F = ${output.toStringAsFixed(2)} °C";
      break;
    case "Liters → Gallons":
      output = input * 0.264172;
      _result = "$input L = ${output.toStringAsFixed(2)} gallons";
      break;
    case "Gallons → Liters":
      output = input / 0.264172;
      _result = "$input gallons = ${output.toStringAsFixed(2)} L";
      break;
    default:
      _result = " Unknown conversion!";
  }
}

```

```

    setState(() {});
  }

  void _clear() {
    _inputController.clear();
    setState(() {
      _result = "";
    });
  }

  void _copyResult() {
    if (_result.isNotEmpty) {
      Clipboard.setData(ClipboardData(text: _result));
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text(" Result copied to clipboard")),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.title),
        backgroundColor: Colors.indigo,
        foregroundColor: Colors.white,
        centerTitle: true,
      ),
      body: Container(
        decoration: const BoxDecoration(
          gradient: LinearGradient(
            colors: [Colors.indigo, Colors.blueAccent],
            begin: Alignment.topLeft,
            end: Alignment.bottomRight,
          ),
        ),
        child: Center(
          child: SingleChildScrollView(
            padding: const EdgeInsets.all(20),
            child: Card(
              elevation: 12,
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(20),
              ),
              child: Padding(
                padding: const EdgeInsets.all(25),
                child: Column(
                  mainAxisAlignment: MainAxisAlignment.min,
                  children: [

```

```

TextField(
  controller: _inputController,
  keyboardType: TextInputType.number,
  decoration: InputDecoration(
    labelText: "Enter value",
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(12),
    ),
  ),
),
const SizedBox(height: 20),

DropdownButtonFormField<String>(
  value: _selectedConversion,
  isExpanded: true,
  items: widget.conversions
    .map((c) => DropdownMenuItem(value: c, child: Text(c)))
    .toList(),
  onChanged: (value) {
    setState(() {
      _selectedConversion = value!;
    });
  },
  decoration: InputDecoration(
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(12),
    ),
  ),
),
const SizedBox(height: 20),

Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: [
    ElevatedButton.icon(
      onPressed: _convert,
      icon: const Icon(Icons.swap_horiz),
      label: const Text("Convert"),
    ),
    ElevatedButton.icon(
      onPressed: _clear,
      icon: const Icon(Icons.clear),
      label: const Text("Clear"),
      style: ElevatedButton.styleFrom(
        backgroundColor: Colors.redAccent,
      ),
    ),
  ],
),
const SizedBox(height: 20),

```

```

if (_result.isNotEmpty)
  Column(
    children: [
      Text(
        _result,
        textAlign: TextAlign.center,
        style: const TextStyle(
          fontSize: 18,
          fontWeight: FontWeight.bold,
        ),
      ),
      const SizedBox(height: 10),
      ElevatedButton.icon(
        onPressed: _copyResult,
        icon: const Icon(Icons.copy),
        label: const Text("Copy Result"),
      ),
    ],
  ),
),
),
),
),
),
),
),
),
),
);
}
}

```

OUTPUT

← → ↻ localhost:64008

Unit Converter

Kilometers → Miles ▼

Enter value
2

Convert

Result: 1.243

Result:

Thus the cross platform application to convert units from imperial system to metric system (Km to miles) developed and executed successfully

Aim:-

To develop a cross platform application for day to day task (to-do) management

Procedure:-**Flutter Installation**

Download flutter SDK from the official site
<https://flutter.dev/docs/get-started/install/windows>

Download the latest Flutter SDK ZIP.

Extract it to a folder (e.g., C:\src\flutter).

Add Flutter to PATH

Open Start → Edit the system environment variables → Environment Variables.

Under “User variables”, find or create Path → Click Edit → New.

Add the path:C:\src\flutter\bin

Click OK to save.

Verify Installation

Open Command Prompt and run:

```
flutter doctor
```

This checks your environment and displays any missing dependencies.

open vs code :

```
flutter create unit_converter
```

```
cd unit_converter
```

```
code .
```

change the code in the lib/main.dart

To run the application

```
flutter run
```

Program

```
import 'package:flutter/material.dart';

void main() {
  runApp(const TodoApp());
}

class TodoApp extends StatelessWidget {
  const TodoApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'To-Do App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
        useMaterial3: true,
      ),
      home: const TodoHomePage(),
    );
  }
}

class TodoHomePage extends StatefulWidget {
  const TodoHomePage({super.key});

  @override
  State<TodoHomePage> createState() => _TodoHomePageState();
}

class _TodoHomePageState extends State<TodoHomePage> {
  final List<Map<String, dynamic>> _tasks = [];
  final TextEditingController _controller = TextEditingController();

  void _addTask() {
    final text = _controller.text.trim();
    if (text.isEmpty) return;

    setState(() {
      _tasks.add({'title': text, 'isDone': false});
      _controller.clear();
    });
  }

  void _toggleTask(int index) {
    setState(() {
      _tasks[index]['isDone'] = !_tasks[index]['isDone'];
    });
  }
}
```



```

void _deleteTask(int index) {
  setState(() {
    _tasks.removeAt(index);
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('My To-Do List'),
      centerTitle: true,
    ),
    body: Column(
      children: [
        // Input field
        Padding(
          padding: const EdgeInsets.all(16.0),
          child: Row(
            children: [
              Expanded(
                child: TextField(
                  controller: _controller,
                  decoration: const InputDecoration(
                    labelText: 'Enter a new task',
                    border: OutlineInputBorder(),
                  ),
                ),
                onSubmitted: (_) => _addTask(),
              ),
              const SizedBox(width: 8),
              ElevatedButton(
                onPressed: _addTask,
                child: const Text('Add'),
              ),
            ],
          ),
        ),
        // Task list
        Expanded(
          child: _tasks.isEmpty
            ? const Center(
                child: Text(
                  'No tasks yet. Add one!',
                  style: TextStyle(fontSize: 16, color: Colors.grey),
                ),
              )
            : ListView.builder(

```

```

itemCount: _tasks.length,
itemBuilder: (context, index) {
  final task = _tasks[index];
  return Card(
    margin:
      const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
    child: ListTile(
      leading: Checkbox(
        value: task['isDone'],
        onChanged: (_) => _toggleTask(index),
      ),
      title: Text(
        task['title'],
        style: TextStyle(
          decoration: task['isDone']
            ? TextDecoration.lineThrough
            : TextDecoration.none,
          color: task['isDone'] ? Colors.grey : Colors.black,
        ),
      ),
      trailing: IconButton(
        icon: const Icon(Icons.delete, color: Colors.red),
        onPressed: () => _deleteTask(index),
      ),
    ),
  );
},
),
),
);
}
}
}
}

```

Output

What do you want to do today?

buy milk

☒ homework ~~3:00 pm~~

☐ football 5:00pm

☐ dinner 7:30 pm

2 tasks left.

Clear 1 completed task

Result:

Thus the cross platform application for day to day task (to-do) management was developed and executed successfully.